Assignment 2 Report

Question 1

Initially, while I had implemented the code normally and the behavioral simulation produced the expected results, in the post-synthesis timing simulation the output became 1 when I had 2 consecutive ones (aligned at the beginning of the triplets), i.e., it gave 1 for 110 and 111. After a lot of effort, adding registers at the input and changing the reset from synchronous to asynchronous without seeing any result, I realized that essentially when we were in the `D1_is_1` state, before Din could change to 0 (in the case of 110) because in the simulation I changed it at the rising edge of the clock as shown in the slides, there was a metastable state. It saw Din as 1 and the state as `D1_is_1`, set `Err = 1`, transitioned to the `Start` state, and then captured 0 as the next input, resulting in an incorrect output.

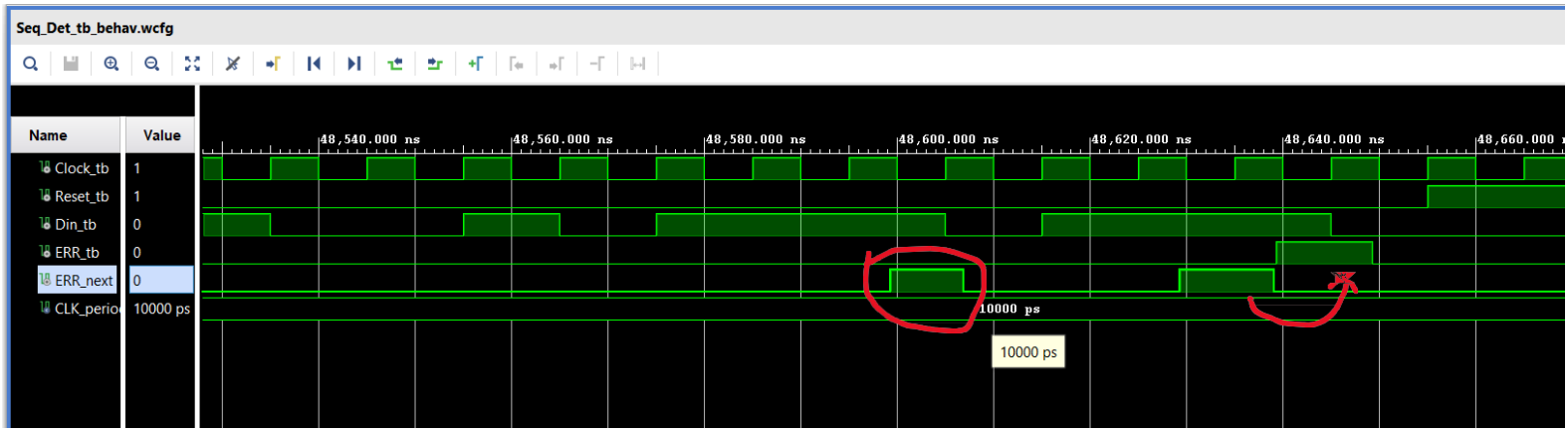
I finally addressed this by placing a register at the output. This way, the state issue is contained within `ERR_next`, and I only output it to the `ERR` signal on the clock's rising edge. As a result, the output is correct.

While searching, I found that this practice is common and highly recommended. Link: https://medium.com/@aiclab.official/finite-state-machines-fsm-aa0e5bf10b03

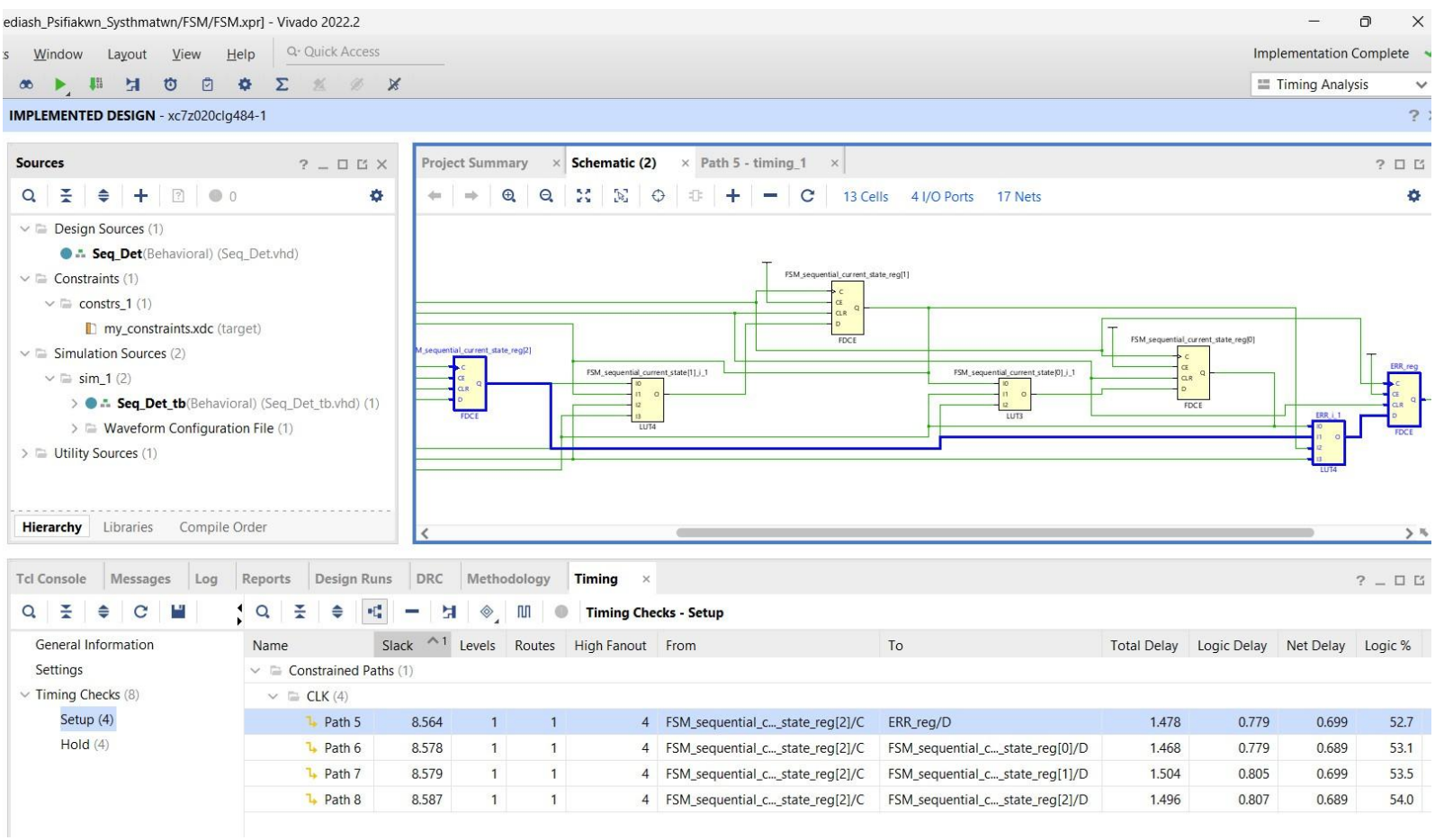
Registered Outputs and Design Partitioning

A recommended approach for partitioning a design is to use combinational logic for input processing and to have registered outputs. For internal signals between modules in an ASIC d Highlight Share ach simplifies writing timing constraints. For external signals in both FPGAs and ASICs, registered outputs produce fewer glitches because they are stable between clock edges. Therefore, in most designs, it is better to use registered outputs and to register the output of the FSM [3].

In the photo below, we can see that `ERR_next` does not propagate to the output in the incorrect case (the 1st one) because the ones arrive as 101, 110. However, in the case where 3 ones are aligned in a triplet (111) and `ERR` must become 1, it passes normally but just appears in the next clock cycle.

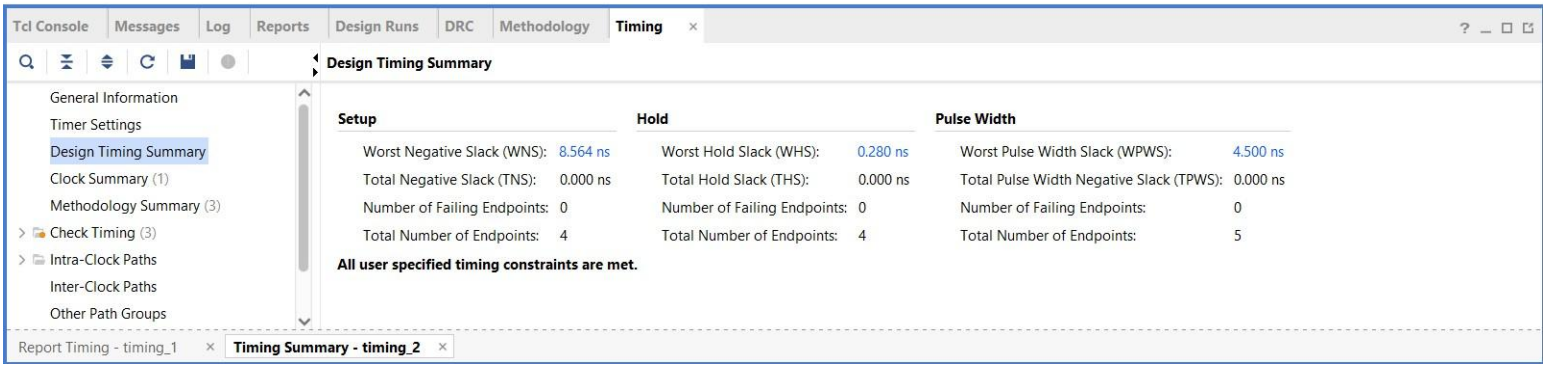


Regarding setup times and circuit speed (Question c):

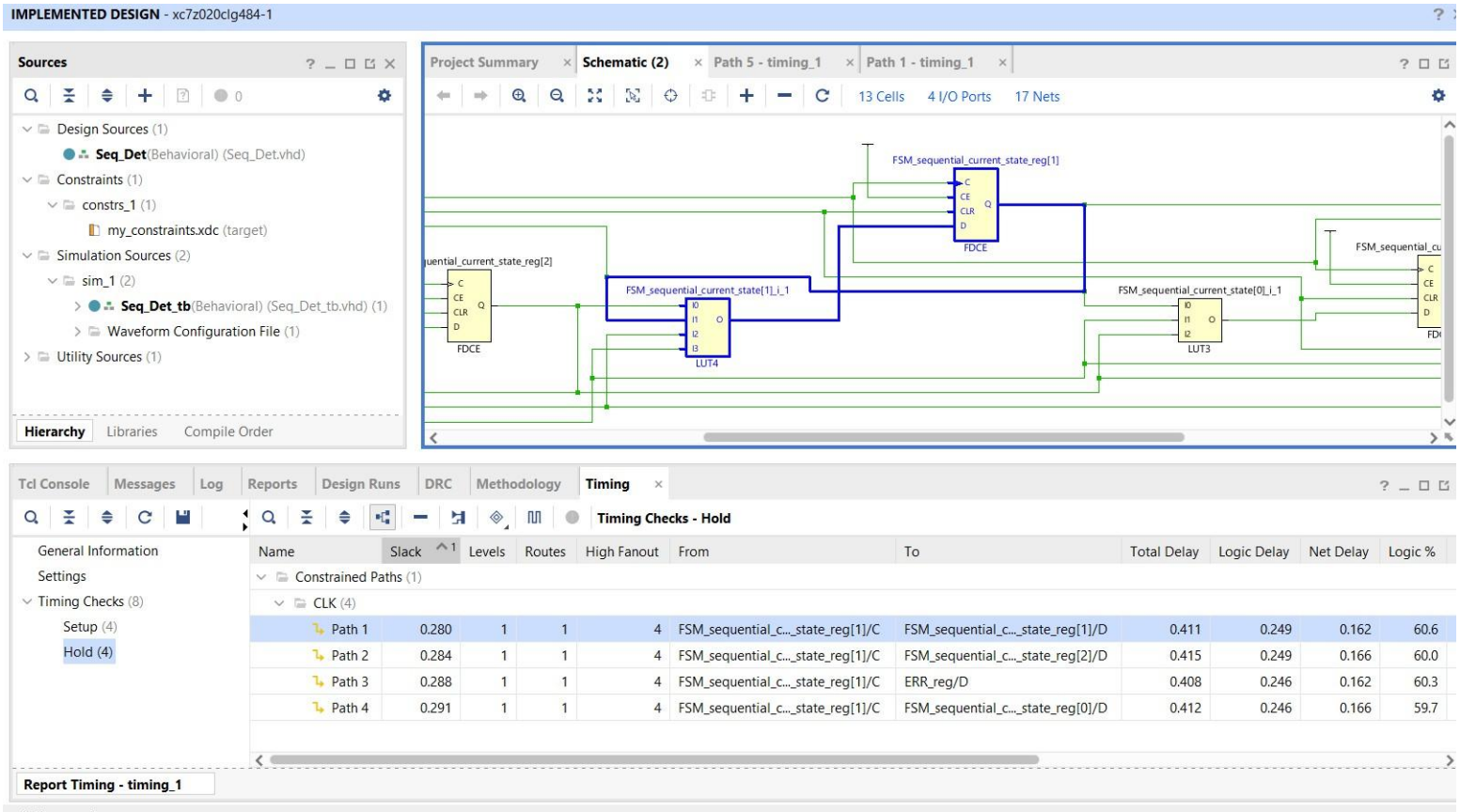


The critical path is path 5, which is the one with the smallest slack. We can observe that path 7 has the largest total logic/net delay, so I investigated and found that the smaller slack is actually a result of skew time and uncertainty time. Nevertheless, these are identical in the two paths, so, as we discussed in the lab, the difference (i.e., a path with a smaller delay having less slack) can even be attributed to the physical placement of the components on the chip.

In any case, the Worst Negative Slack (WNS) = 8.564 ns. Therefore, the minimum clock period for our circuit is 10.000 - 8.564 = 1.436 ns, corresponding to a maximum frequency of 696.37 MHz. One observation is that when I set this value via the edit timing constraints, there was a slight violation of the FPGA's Pulse Width constraints (it appeared as negative in the timing report). So, to be precise, the frequency is further limited slightly by the physical characteristics of the FPGA.

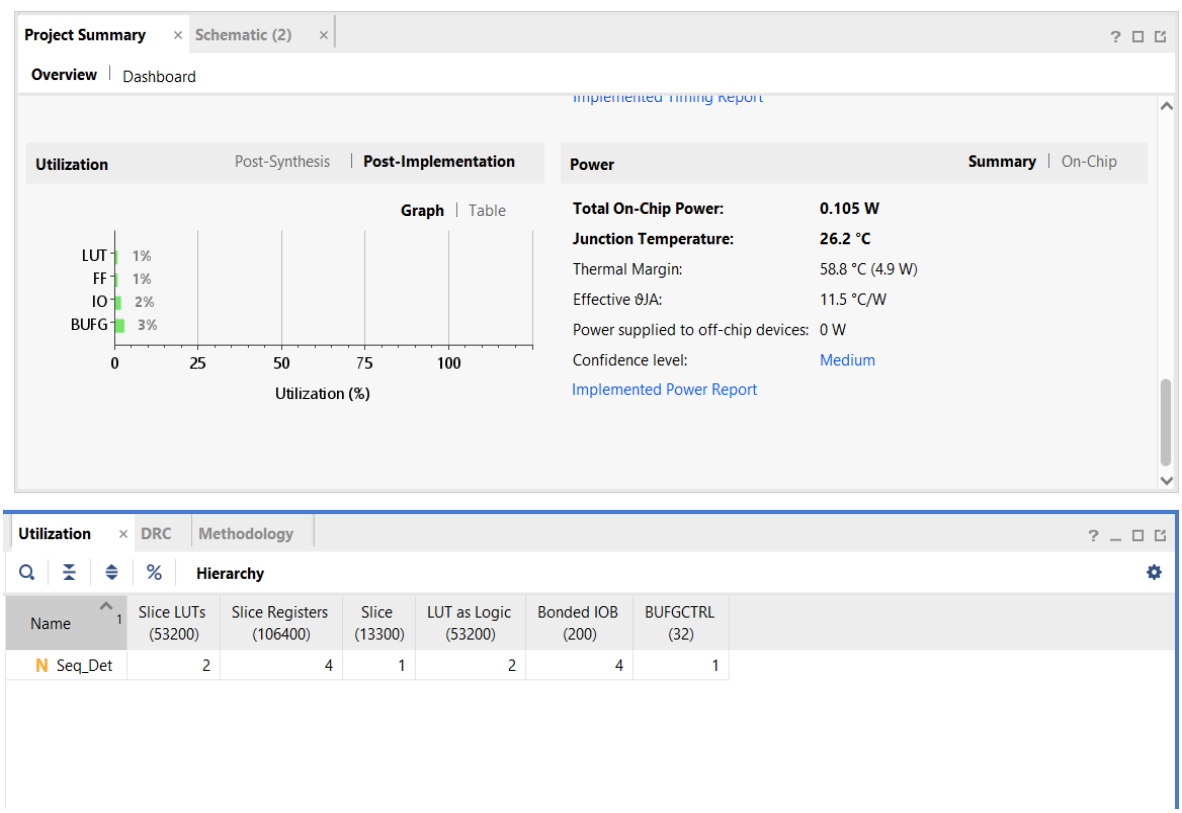


Similarly, the contamination delay corresponds to the path with the smallest slack, which is path 1 with a slack of 0.280 ns and a total delay of 0.411 ns.



Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Logic %
Constrained Paths (1)										
CLK (4)										
Path 1	0.280	1	1	4	FSM_sequential_c..._state_reg[1]/C	FSM_sequential_c..._state_reg[1]/D	0.411	0.249	0.162	60.6
Path 2	0.284	1	1	4	FSM_sequential_c..._state_reg[1]/C	FSM_sequential_c..._state_reg[2]/D	0.415	0.249	0.166	60.0
Path 3	0.288	1	1	4	FSM_sequential_c..._state_reg[1]/C	ERR_reg/D	0.408	0.246	0.162	60.3
Path 4	0.291	1	1	4	FSM_sequential_c..._state_reg[1]/C	FSM_sequential_c..._state_reg[0]/D	0.412	0.246	0.166	59.7

The resources used after implementation are:



Name	Slice LUTs (53200)	Slice Registers (106400)	Slice (13300)	LUT as Logic (53200)	Bonded IOB (200)	BUFGCTRL (32)
Seq_Det	2	4	1	2	4	1

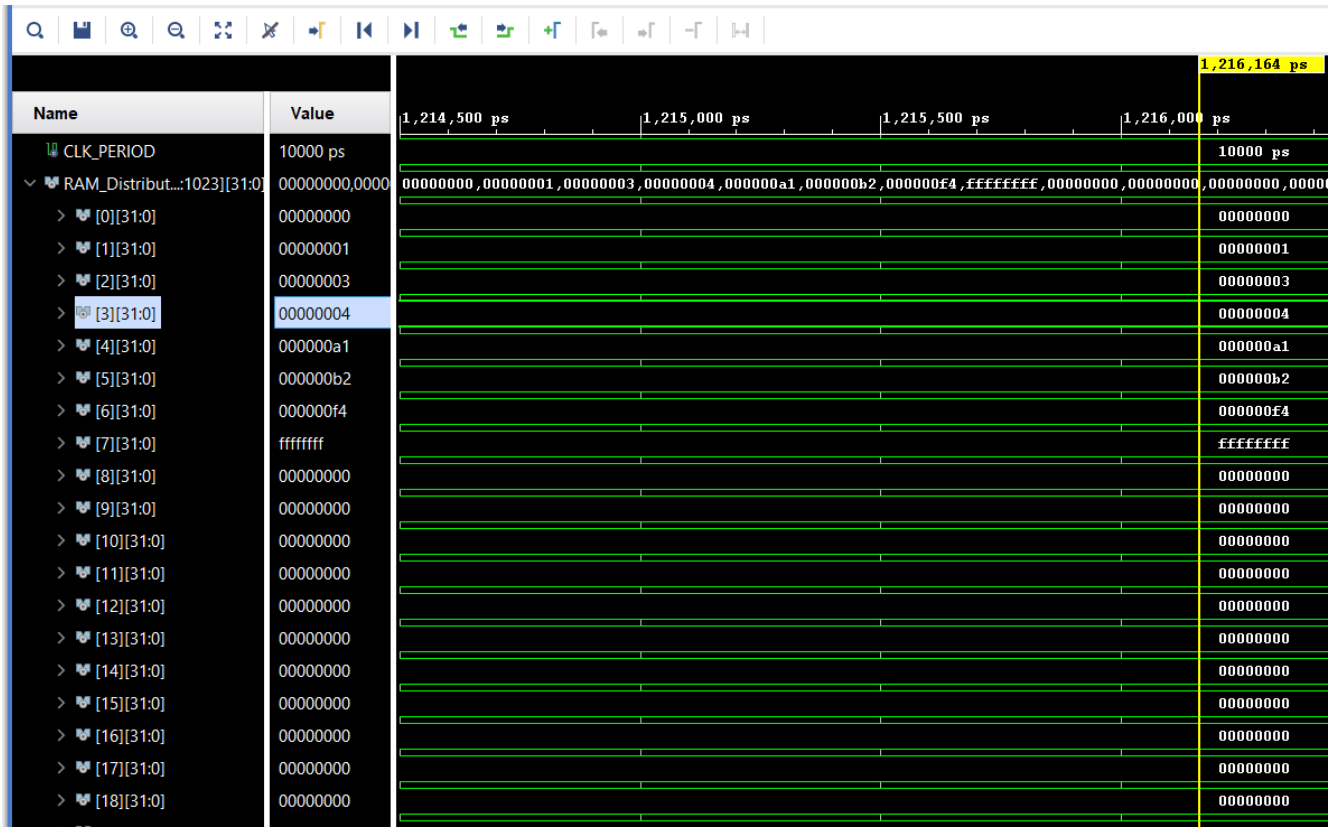
Question 2

Initially, it quickly became apparent that we cannot simply use two loops, as we do in software. The first solution that came to mind was that the algorithm has some common steps and a few groups of steps (compare, next_i, next_j, swap...) which can be translated into FSM states. Another idea I had was to read all values into registers, pass them to variables, and execute the code normally within a process. However, with distributed RAM, we can only read 1 data item at a time (at least with the single-port RAM provided), so some kind of state machine would still be required to read all the values. Furthermore, I was concerned about scalability with this approach; for instance, with 1,000 ROM values, I would need 1,000 variables, resulting in a massive combinational logic block. Ultimately, I settled on the FSM logic.

The state flow is as follows:

- **S_LOAD:** Before sorting begins, we populate the RAM.
- **S_READ_1, S_READ_2:** Since we cannot read two addresses simultaneously from a simple RAM, we split the read operation into two cycles.
- **S_COMPARE:** We compare the two registers. If `r_temp_val > r_temp_val_2`, a swap is necessary. If not, we proceed to the next pair (incrementing j) or the next iteration (incrementing i), simulating the nested `for` loops.
- **S_SWAP_1, S_SWAP_2:** Similar to the read operation, writing the new values must be done in two steps.

A `done` signal is included so we can observe the algorithm's completion in the simulation. Regarding the simulation, I initially added the RAM array to the Behavioral simulation, as shown in the image below. I simply ran the algorithm and monitored the internal `ram_distributed` signal, visually verifying if the values were sorted.



However, in the post-synthesis timing simulation, this RAM structure is inaccessible in the waveforms. The RAM is synthesized into LUTs, making it extremely difficult, if not impossible, to determine which LUTs hold which memory addresses and values. Therefore, in the simulation deliverable, there is a constant array containing the ROM inputs. These are sorted into variables within the process, and we simply wait for the sorting to finish to verify their correctness.

At first, I considered accessing the RAM array through the simulation code using something like `uut.ram_distributed`, but upon searching, I realized this is only available in VHDL 2008 and is known as Hierarchical References (External Names). Next, I thought about adding an extra state (after `S_DONE`) to iterate through the memory and output each location from the RAM for verification. However, this would increase the states and hardware logic solely for simulation purposes. Finally, I chose an intermediate solution:

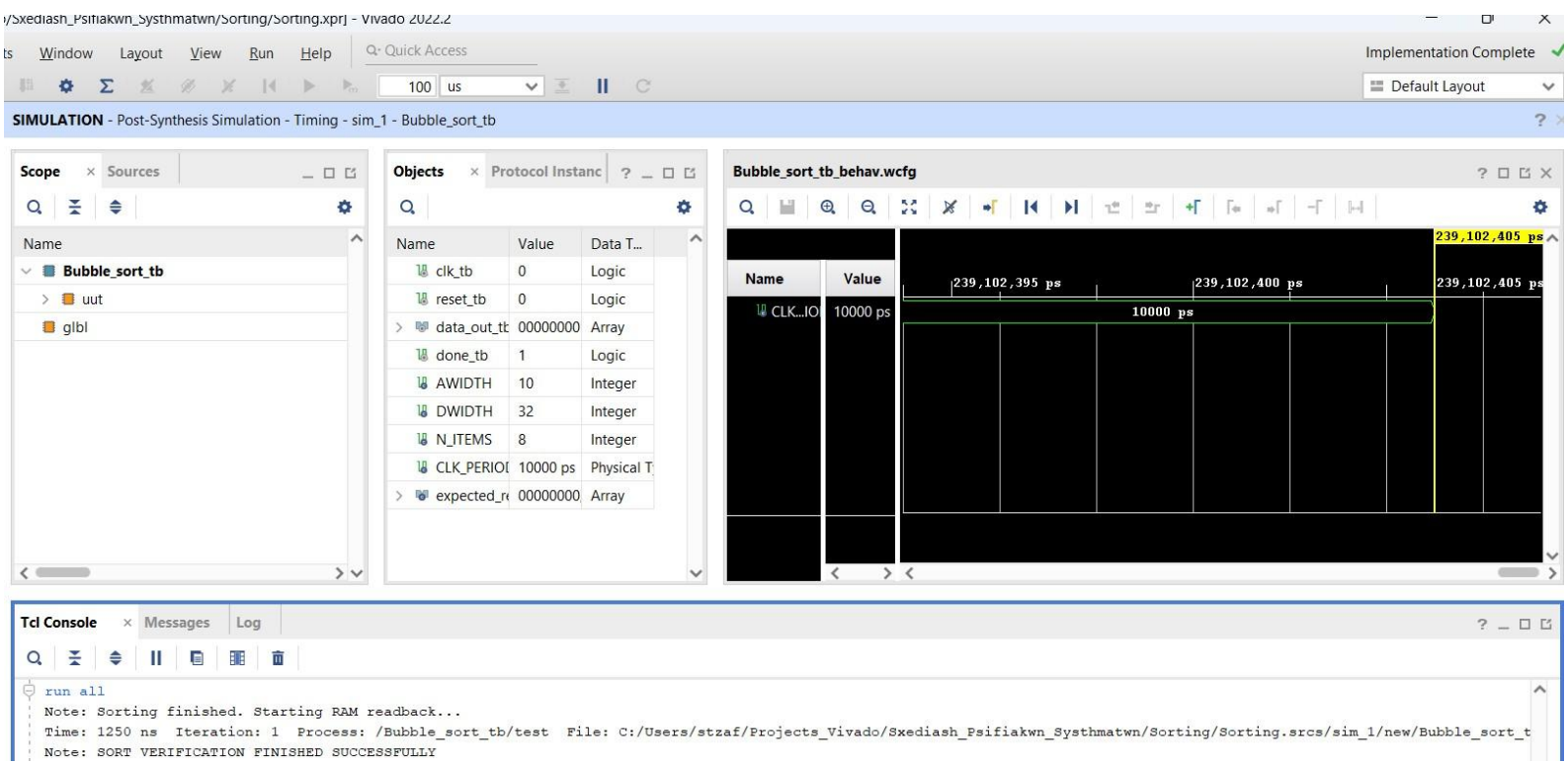
I did not add a new state, only the `done` signal. Within the `S_DONE` state, I iterate through each memory location once. This provides the ability to verify the memory

contents as expected. Furthermore, implementing the memory scan within `S_DONE` does not consume additional FPGA resources, as the synthesizer likely reuses the `r_i` counter already required for the initial load phase (`S_LOAD`). Thus, we gain the ability to read the results at no extra hardware cost.

Lastly, because it was still difficult to manually inspect the `Data_Out` results and determine when the sorting finished just by looking at the waveform, I added a loop in the simulation code. This loop starts as soon as `done` becomes 1 (sorting ends) and synchronously checks (with the clock) if the circuit's output value (`data_out_tb`) matches the value in `expected_results(i)`. If they do not match, it prints "ERROR" in the console, indicating an unsuccessful sort.

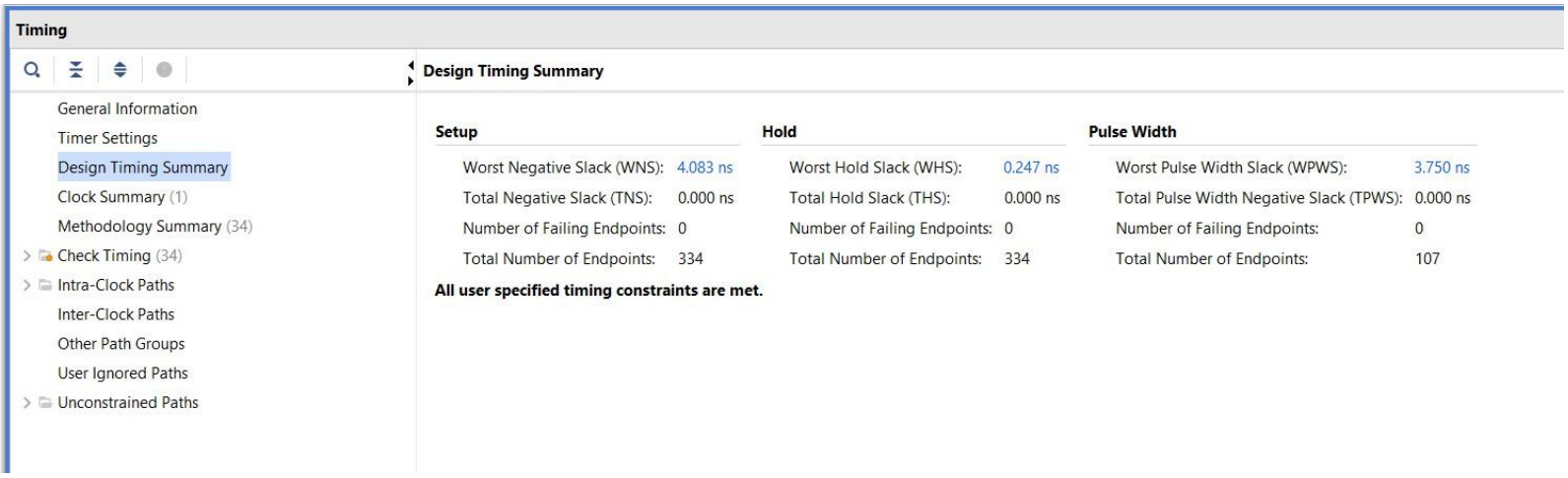
Additionally, the circuit updates the data on the rising edge. The testbench reads the data on the falling edge. This ensures that the data is stable (satisfying setup time) before we evaluate it. When I checked on the rising edge for both, I received errors, and the console messages indicated it was reading a previous value.

To run the simulation and cleanly view the messages, I added a 100 sec wait after the first execution. So, if you hit "run simulation", then "clear console", and run again, the following output will appear:

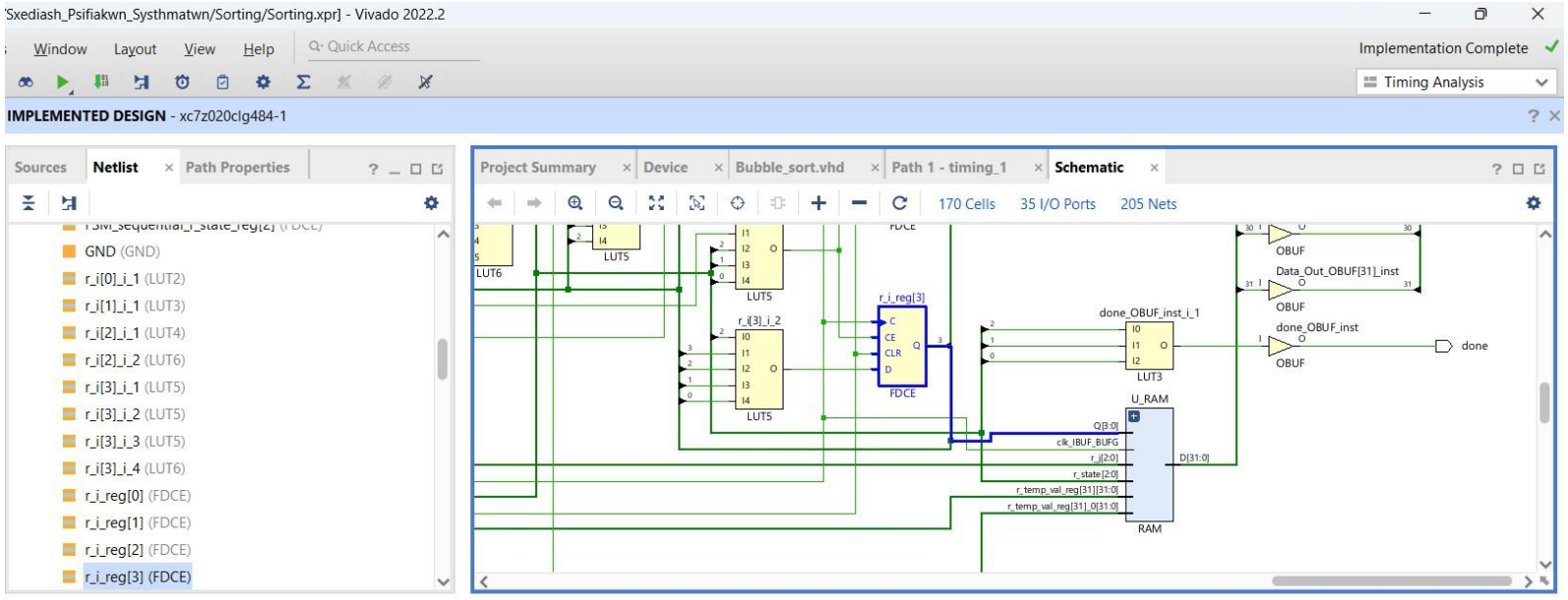


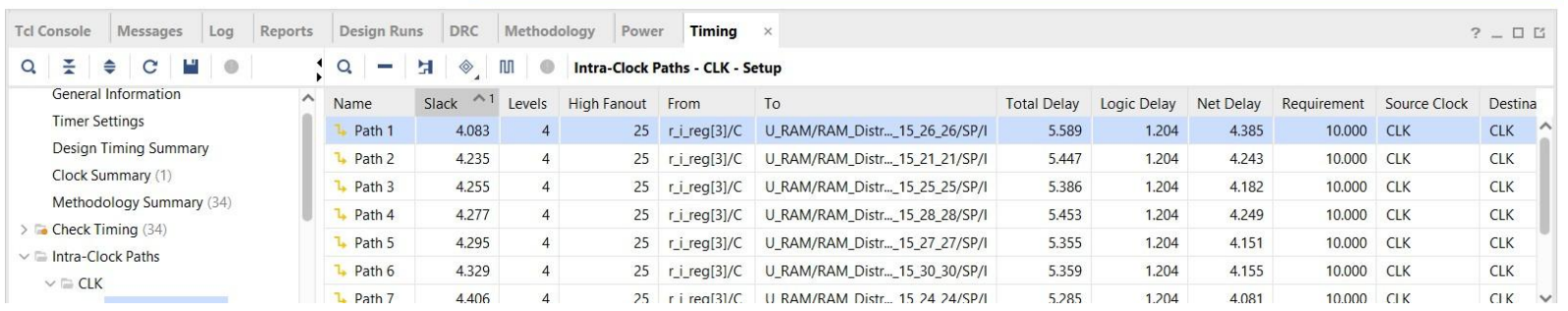
A minor clarification for the simulation: if we want to change the values in the ROM, we must also update the constant array input in the simulation. Similarly, if the number of elements changes, the `N_ITEMS` constant must be updated in both `Bubble_sort.vhd` and `Bubble_sort_tb.vhd`.

Times - Resources:



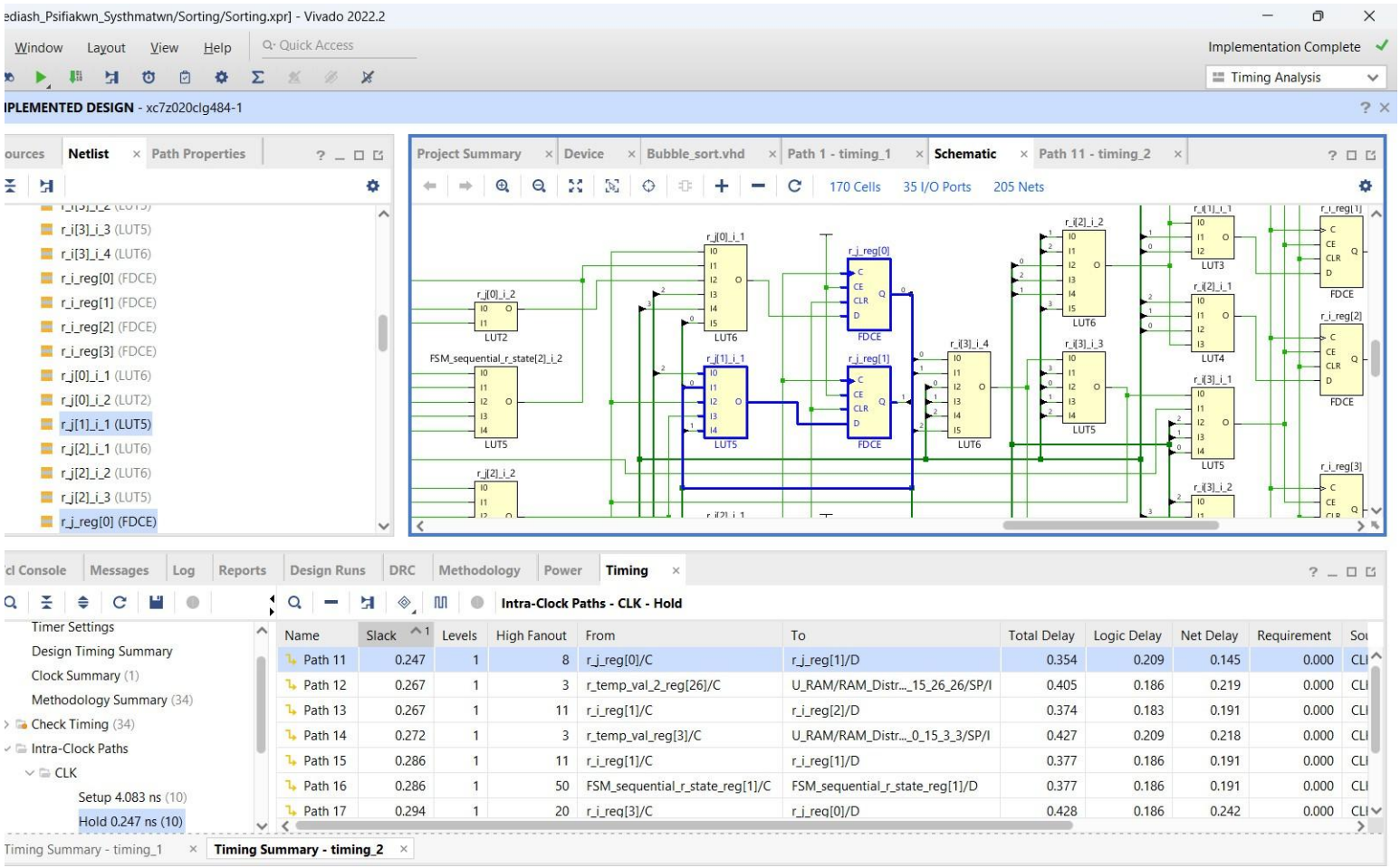
- WNS (Worst Negative Slack): 4.083 ns.
- Therefore, the minimum period is T = 10 - 4.083 = 5.917 ns, with a maximum frequency of F = 169 MHz.
- The critical path is path 1 with a delay of 5.589 ns.



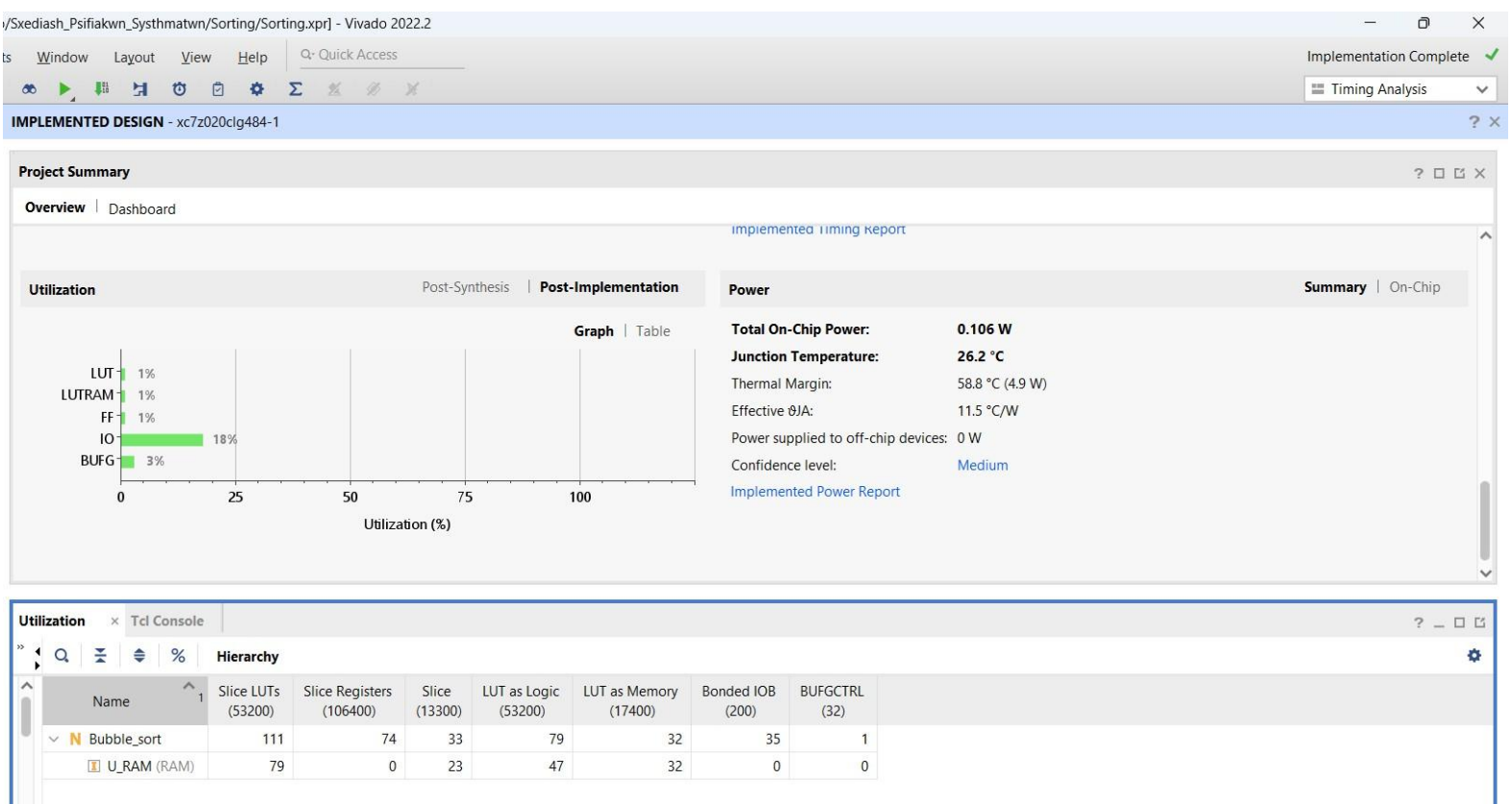


Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destina
Path 1	4.083	4	25	r_i_reg[3]/C	U_RAM/RAM_Distr..._15_26_26/SP/I	5.589	1.204	4.385	10.000	CLK	CLK
Path 2	4.235	4	25	r_i_reg[3]/C	U_RAM/RAM_Distr..._15_21_21/SP/I	5.447	1.204	4.243	10.000	CLK	CLK
Path 3	4.255	4	25	r_i_reg[3]/C	U_RAM/RAM_Distr..._15_25_25/SP/I	5.386	1.204	4.182	10.000	CLK	CLK
Path 4	4.277	4	25	r_i_reg[3]/C	U_RAM/RAM_Distr..._15_28_28/SP/I	5.453	1.204	4.249	10.000	CLK	CLK
Path 5	4.295	4	25	r_i_reg[3]/C	U_RAM/RAM_Distr..._15_27_27/SP/I	5.355	1.204	4.151	10.000	CLK	CLK
Path 6	4.329	4	25	r_i_reg[3]/C	U_RAM/RAM_Distr..._15_30_30/SP/I	5.359	1.204	4.155	10.000	CLK	CLK
Path 7	4.406	4	25	r_i_reg[3]/C	U_RAM/RAM_Distr..._15_24_24/SP/I	5.285	1.204	4.081	10.000	CLK	CLK

Similarly, for contamination times, path 11 is the limiting path, providing a delay of 0.354 ns and a slack of 0.247 ns.



Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement
Path 11	0.247	1	8	r_j_reg[0]/C	r_j_reg[1]/D	0.354	0.209	0.145	0.000
Path 12	0.267	1	3	r_temp_val_2_reg[26]/C	U_RAM/RAM_Distr..._15_26_26/SP/I	0.405	0.186	0.219	0.000
Path 13	0.267	1	11	r_i_reg[1]/C	r_i_reg[2]/D	0.374	0.183	0.191	0.000
Path 14	0.272	1	3	r_temp_val_reg[3]/C	U_RAM/RAM_Distr..._0_15_3_3/SP/I	0.427	0.209	0.218	0.000
Path 15	0.286	1	11	r_i_reg[1]/C	r_i_reg[1]/D	0.377	0.186	0.191	0.000
Path 16	0.286	1	50	FSM_sequential_r_state_reg[1]/C	FSM_sequential_r_state_reg[1]/D	0.377	0.186	0.191	0.000
Path 17	0.294	1	20	r_i_reg[3]/C	r_j_reg[0]/D	0.428	0.186	0.242	0.000



Name	Slice LUTs (53200)	Slice Registers (106400)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	Bonded IOB (200)	BUFGCTRL (32)
Bubble_sort	111	74	33	79	32	35	1
U_RAM (RAM)	79	0	23	47	32	0	0